

Behavioral Synthesis of ICs

Introduction to HLS

Credentials: majority of information are from the textbook:

Gajski, Daniel D., Nikil D. Dutt, Allen CH Wu, and Steve YL Lin. *High—Level Synthesis: Introduction to Chip and System Design*. Springer Science & Business Media, 2012.

Outline

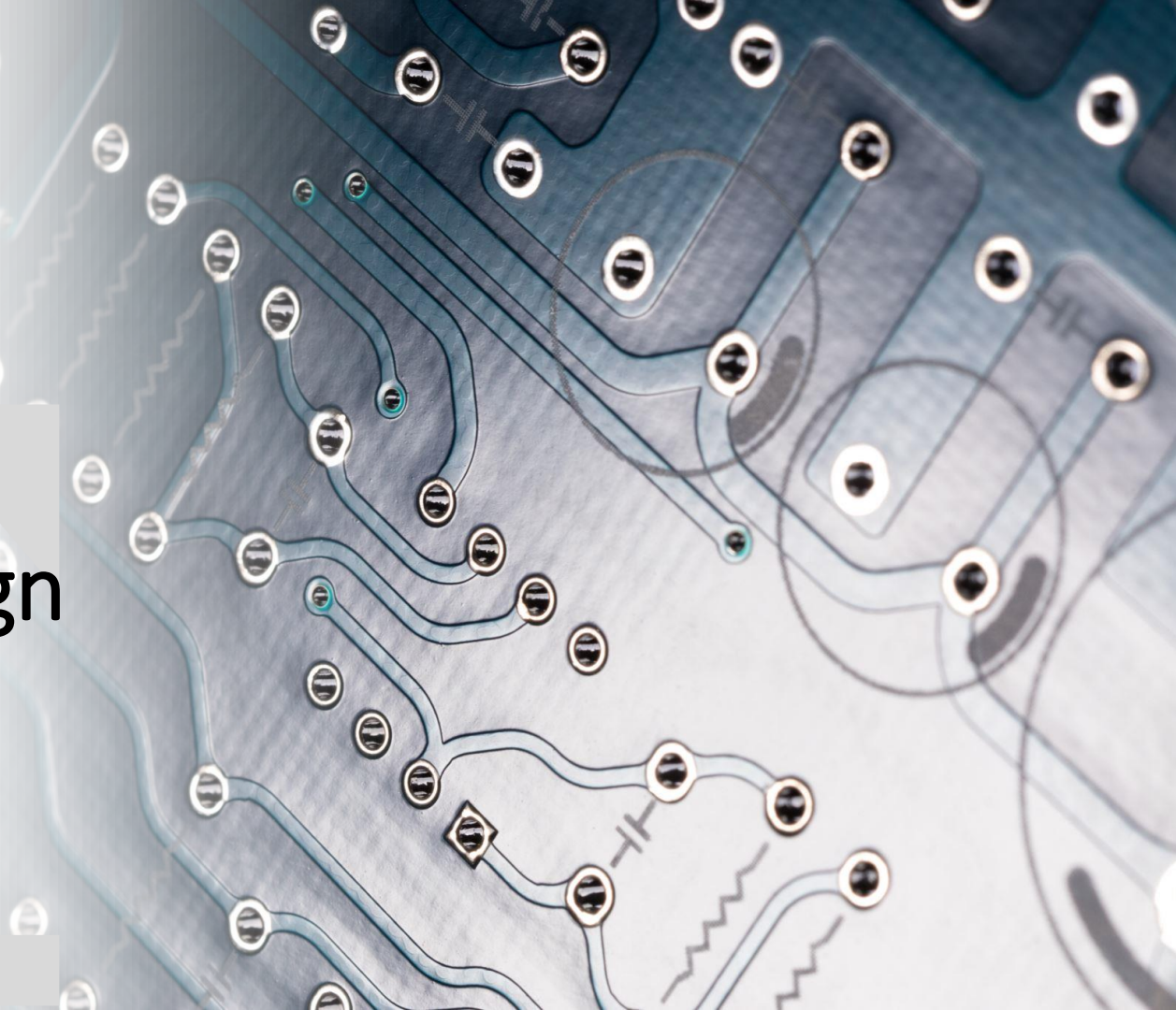
- The Need for Design Automation on Higher Abstraction Levels
- Levels of Abstraction
- Definition of Synthesis
- Languages, Designs and Technologies
- Essential Issues in Synthesis
 - High-level Formulation
 - Languages and Representations
 - Design Modeling
 - Design Quality-Measures
 - High-Level Synthesis Algorithms
 - Component Sets and Technology Mapping
 - Databases and Environments
- Status and Future of High-Level Synthesis: ***research assignment***

Objectives

1. To introduce the concept, importance, and scope of Behavioral Synthesis.
2. Set the course's academic context and significance within the broader landscape of IC design.
3. Lay the foundation for understanding how Behavioral Synthesis improves upon traditional RTL design methods.
4. To be engaged by discussing real-world applications and the evolution of IC design methodologies.
5. Provide a roadmap for the course, detailing the structure, content, resources, and assessment methods.

Current Trends in Integrated Circuit Design

A brief review



Current Trends in Integrated Circuit Design

- VLSI technology reached densities of over **billion transistors** of random logic per chip in the early 2010s.
- Systems of such complexity are very difficult to design by handcrafting each transistor or by defining each signal in terms of logic gates.
- As the complexities of systems increase, so will the need for **design automation** on more abstract levels where functionality and tradeoffs are easier to understand.
- VLSI technology has also reached a maturity level: it is well understood and no longer provides a competitive edge by itself.
- The industry has started looking at the product development cycle comprehensively to increase **productivity** and to gain a competitive edge. Automation of the entire design process **from conceptualization to silicon** has become a necessary goal.

Everything Keeps Getting Faster and Faster!

- Everything does keep getting **faster**.
- Today the computing speed in a throwaway musical birthday card is greater than the fastest mainframes of a few decades ago!
- Multi-Giga-Hertz processors are shipping in mainstream PCs and workstations, and 20 **GHz processors** are running in labs.
- Boards have been successfully designed that transmit 12.5 Gbps over a single differential pair of copper traces, and individual devices are now transmitting data at rates up to 40 Gbps over parallel traces.
- Devices have converged — today, most computing devices communicate and most communication devices compute.
- Billion-transistor processors are produced and the range of devices that can be manufactured in silicon is expanding.
- The question is, how far it can go?

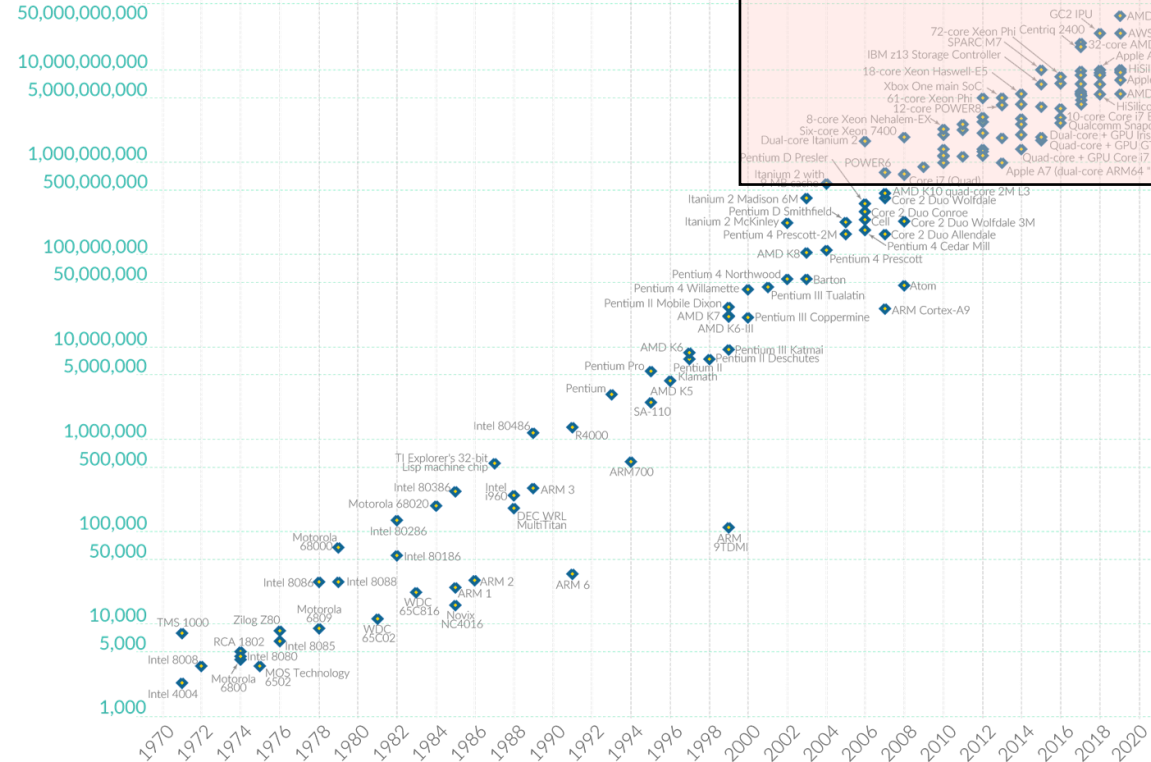
Miniaturization: Moore's Law

- In 1965, just six years after the invention of the integrated circuit, Intel co-founder **Gordon Moore** observed that the **number of transistors** on a given piece of silicon would **double every couple of years** — a profound insight that was dubbed Moore's Law. (exponential growth, do you have any other example in mind?!?)
- Today, Moore's Law remains valid. This exponential growth and ever-shrinking transistor size result in ever-increasing **performance**, **speed**, and **decreased cost**.
- Moore's Law is not a law of science or nature. It is a principle that describes the unique opportunity for exponential improvements provided by advances in semiconductor technology. What began as an observation has become a compass and engine, setting the bar for the semiconductor industry and producing an expanding universe of new applications and opportunities at higher and higher speeds.

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Transistor count



Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Year in which the microchip was first introduced

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Transistor count

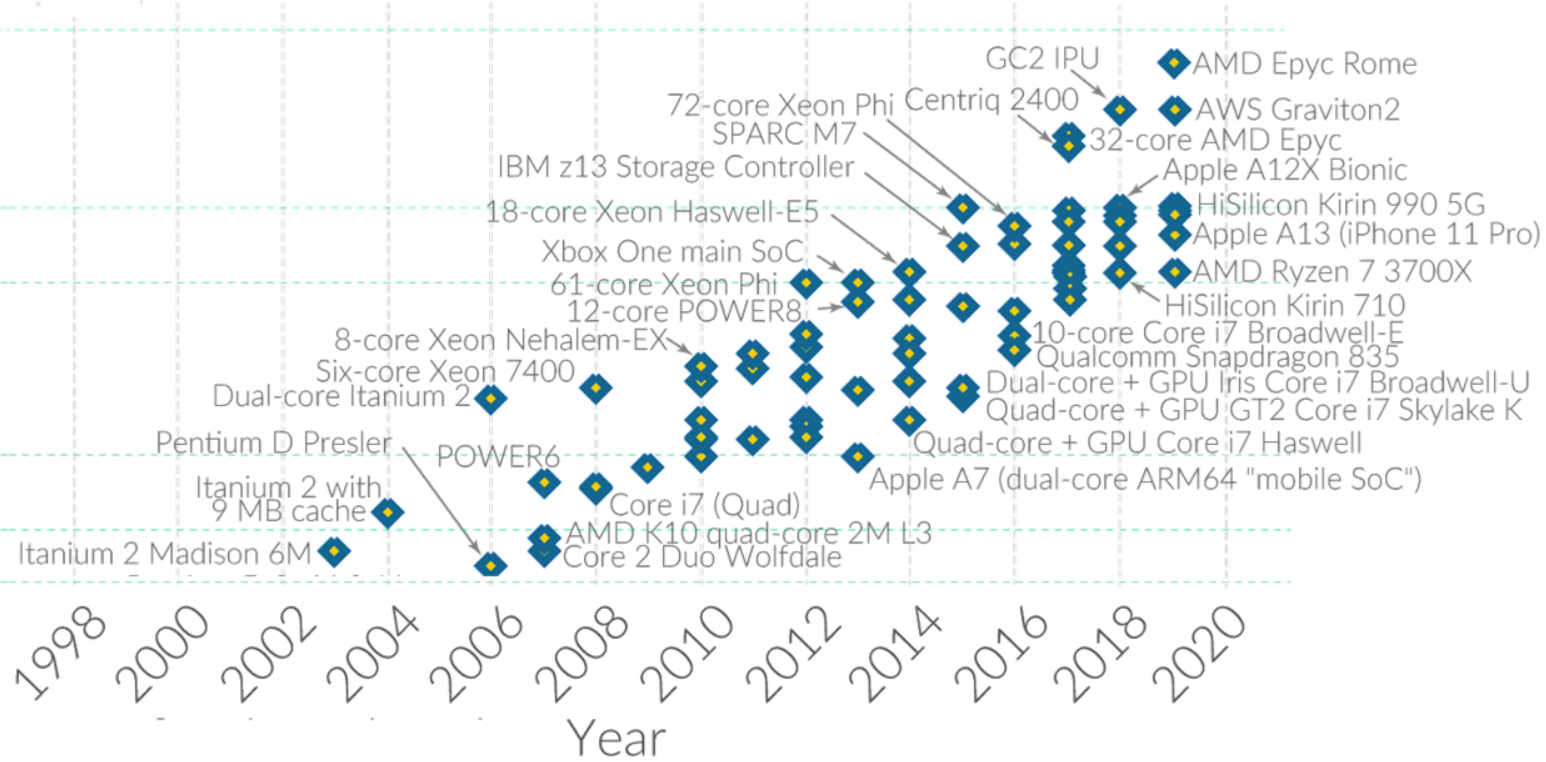
50,000,000,000

10,000,000,000

5,000,000,000

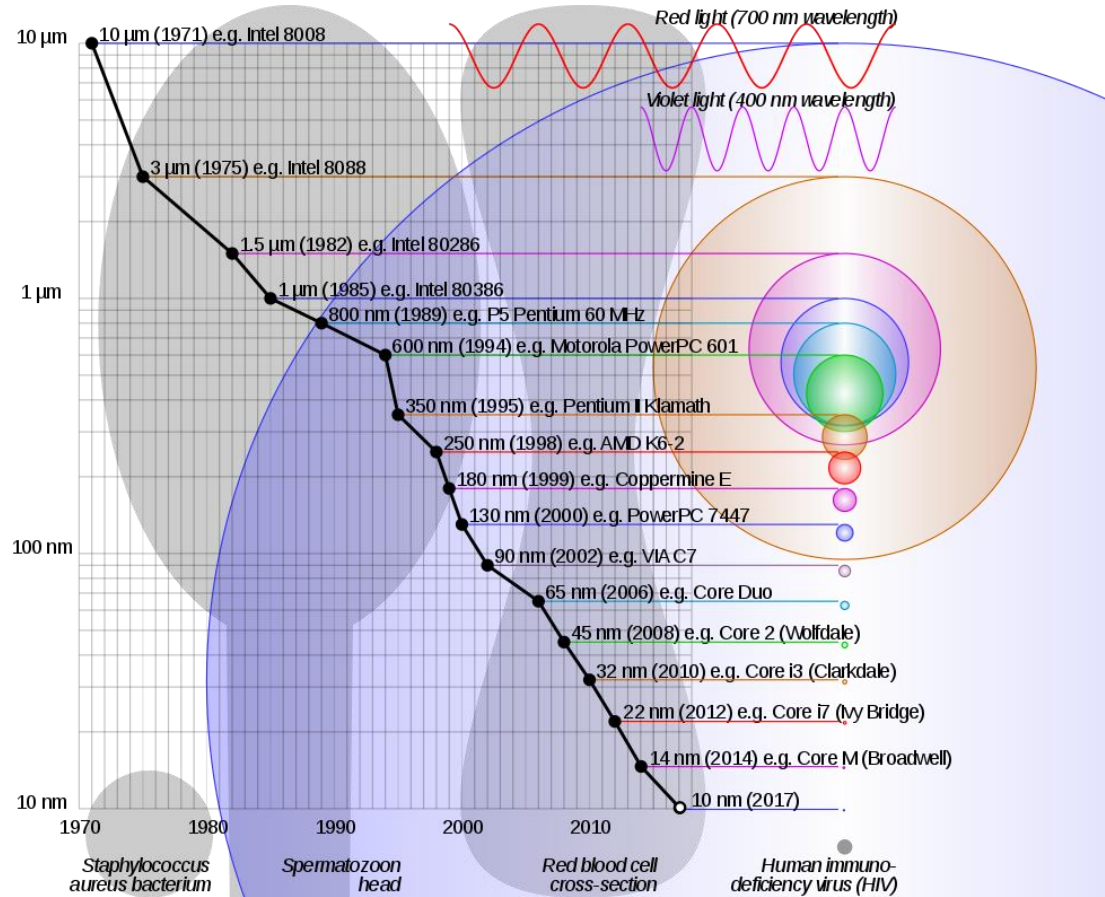
1,000,000,000

500,000,000



Miniaturization

- Progress of miniaturization, and comparison of sizes of semiconductor manufacturing process nodes with some microscopic objects and visible light wavelengths.



Source: Wikipedia, This image is licensed under the [Creative Commons Attribution-Share Alike 3.0 Unported](#) license.

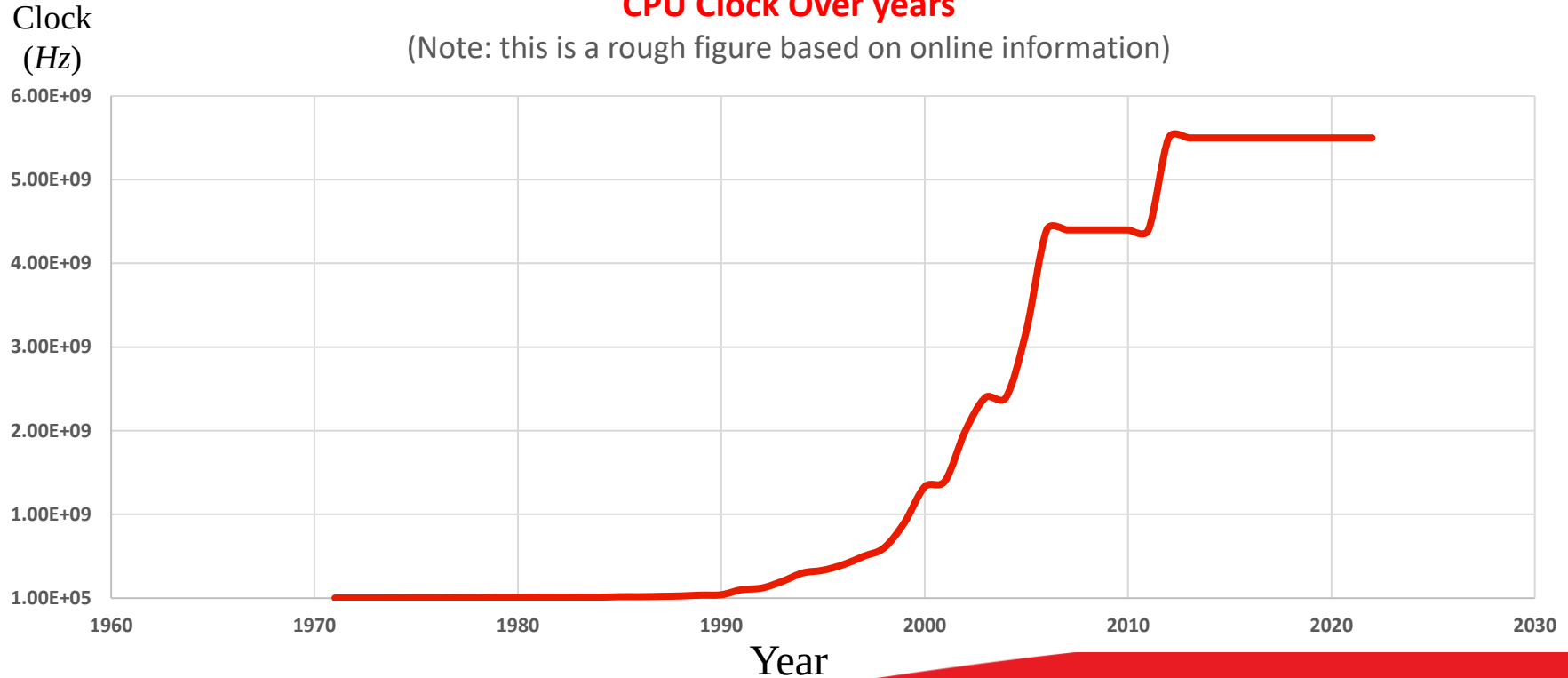
Miniaturization: Clock Speed Ceiling

- Over the decade, transistor counts increased by about an order of magnitude, a trend continued from previous decades and nearly all parameters of microprocessor technology improve as **transistor counts** climb. For example, **speed** and performance have climbed even more sharply than the number of transistors.
- The first microprocessors were designed and manufactured in the 1970s. **Intel's 4004 of 1971** is widely regarded as the first commercial microprocessor, with a clock frequency of **740 kHz**, where the **Zen 3** (AMD) and **M1** (Apple) processors run at **3+ GHz**.
- Excited by transistor count increase, it was predicted that billion-transistor processor would run at speeds approaching to 20 GHz, and engineers would increase processors' clock frequency at the rate of 25 MHz a week and even 25 MHz in a single day!
- But the bitter reality is microprocessor clock speeds had reached a ceiling and has been more or less fixed over years. (**Can you explain why?**)
- Instead of increasing clock frequency, designers have turned to parallel computing in the form of the multi-core processor.

CPUs Clock Frequency Evolution

CPU Clock Over years

(Note: this is a rough figure based on online information)



Computer Aided Design

CAD Tools



Current Trends in Integrated Circuit Design: CAD Tools

- Computer-Aided Design (CAD) research, and the CAD industry in particular, has been very successful and has enjoyed exceptional growth, paralleled only by the advances in IC fabrication.
- Since problems at lower levels of design became **humanly intractable** and time consuming earlier than on higher abstraction levels, CAD researchers and the industry first turned to problems such as **circuit simulation**, **placement**, **routing** and **floorplanning**. CAD tools for **logic simulation** and **synthesis** came later.
- As **design complexities** grew and **time-to-market** requirements shrank drastically, industry and academia started focusing on even higher levels of design than logic and layout. A **higher level of abstraction** reduces the number of objects that a designer needs to consider by an order of magnitude, which in turn allows the design and manufacture of larger systems in shorter periods of time.
- **High-level synthesis is thus the natural next step in the design methodology of VLSI systems.**

Current Trends in Integrated Circuit Design: CAD Tools

- Another reason for the emphasis on high-level design methodologies is that high-level abstractions are closer to a **designer's way of thinking!**
- It is difficult to imagine a designer specifying, documenting and communicating a chip design in terms of a circuit schematic with 1000,000 gates, or a logic description with 1000,000 Boolean expressions!?! 😞
- With increasing design complexity, it becomes impossible for a designer to comprehend the functionality of a chip, or a system specified completely with circuit or **logic schematics**.
- A system described in terms of **higher level components** (e.g., memories, registers, ALUs and buses) and specified using higher level operations on data values over time exposes the design's **functionality** and allows a designer to consider alternative implementations with ease.

Behavioral Synthesis

There are several **advantages** to automating part or all of the design process and moving automation to higher levels:

- Automation assures a much **shorter design cycle**,
 - It allows for more **exploration of different design styles** since different designs can be generated and evaluated quickly.
 - If synthesis algorithms are well understood(?!), design automation tools may outperform average human designers in generating **high quality designs**. However, **correctness verification** of these algorithms, and of CAD tools in general, is not easy.
- CAD tools still cannot match human quality in automation of the entire design process, although human quality can be achieved on a single task. The two main obstacles are
- a large problem size that requires **very efficient search of the design space**, and
 - a detailed design model that requires sophisticated algorithms capable of satisfying **multiple goals** and **multiple constraints**.

Behavioral Synthesis: Classic Approaches

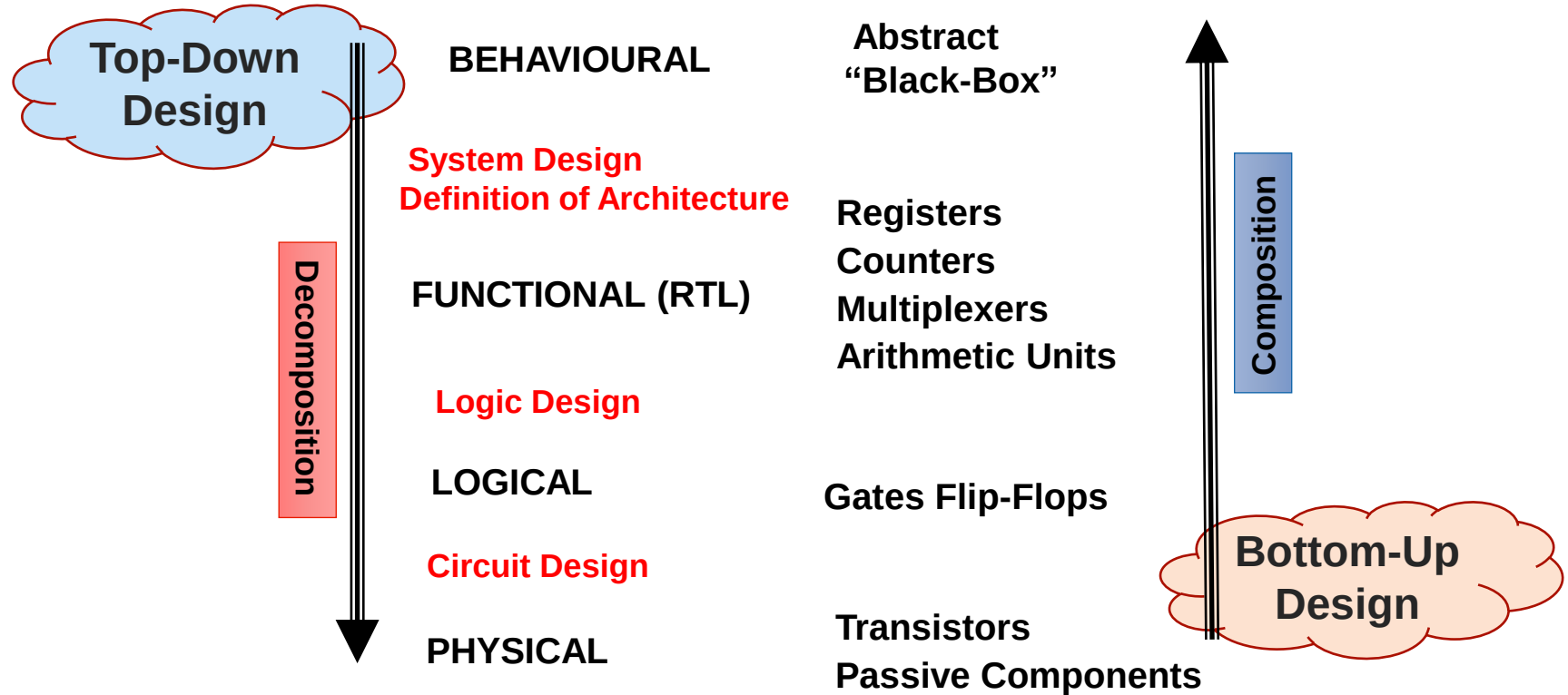
Two schools of thought emerged from the controversy over solutions to these two problems.

- The **capture-and-simulation** school believes that human designers have very good design knowledge accumulated through experience that cannot be automated.
- This school believes that a designer builds a design hierarchy in a **bottom-up fashion** from elementary components such as transistors and gates.
- In this approach, design automation should provide CAD tools that capture various aspects of design and verify them **predominantly through simulation**.
- This approach focuses on a common framework that **integrates capture-and-simulation tools** for different levels of abstraction and allows designers to move easily between different levels.

Behavioral Synthesis: Classic Approaches (continued)

- The **describe-and-synthesize** school believes that synthesis algorithms can out-perform human designers.
- This approach assumes that human designers optimize a small number of objects well but are not capable of finding an optimal design when thousands of objects are in question. CAD algorithms search the design space more thoroughly and are capable of finding nearly optimal designs.
- This school believes that a **top-down methodology**, in which designers describe the intent of the design and CAD tools add detailed physical and electrical structure, would be better suited for the future design of complex systems.
- This approach focuses on definition of description **languages**, **design models**, **synthesis algorithms** and environments for interactive synthesis in which designers' intuition can substantially reduce the search through the design space.

Generic Design Flow



Behavioral Synthesis: Classic Approaches (continued)

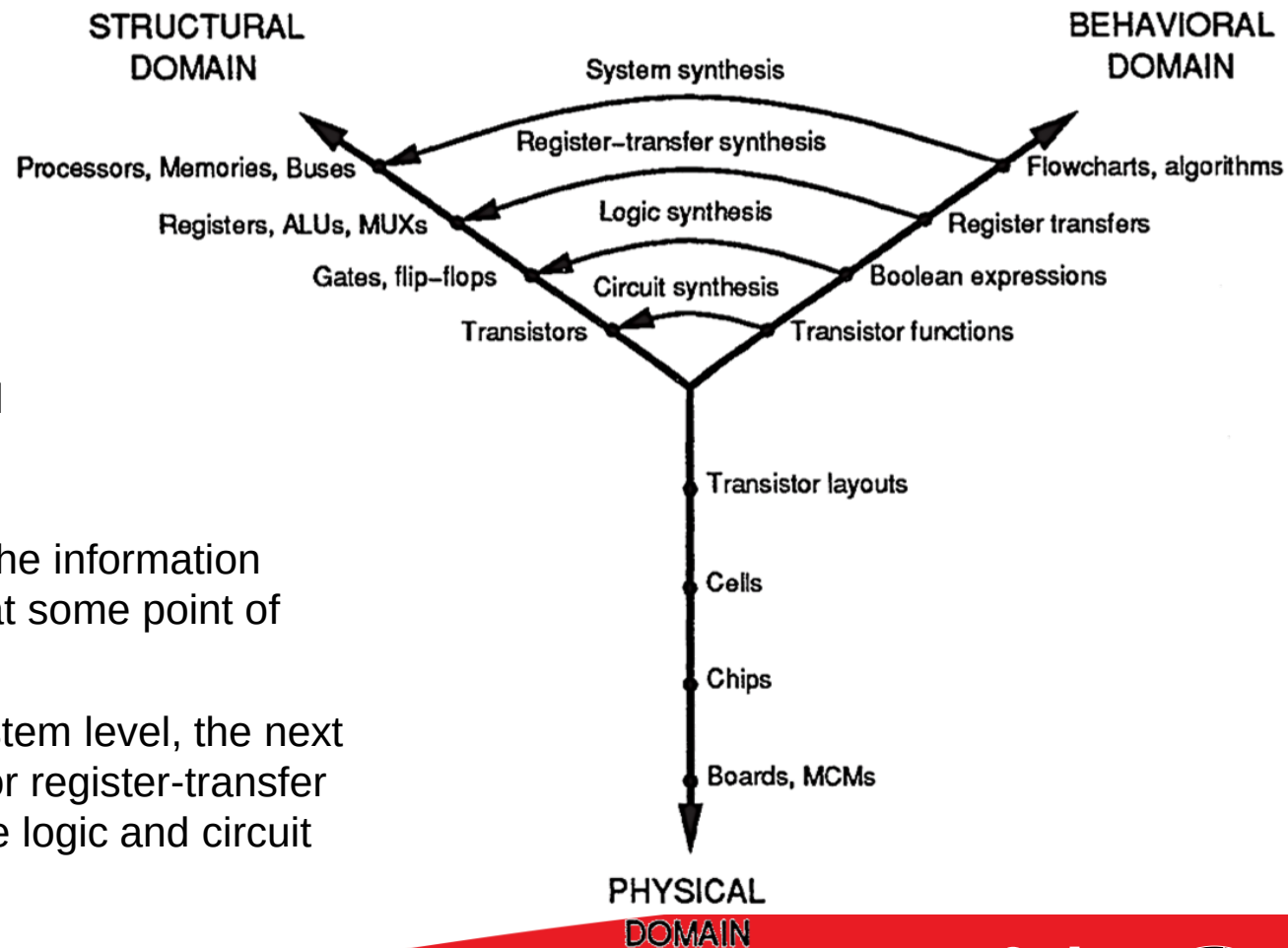
- Both these schools of thought may be correct at some point during the evolution of the technology.
 - For example, it is still profitable to handcraft a memory cell that is replicated millions of times.
 - On the other hand, optimizing a 100,000-gate design while using a gate-array with 1,000,000 gates is not cost effective if the design already satisfies all other constraints. Thus, at this point in technological evolution, even suboptimal synthesis becomes more cost effective than design handcrafting.
- Nowadays, VLSI technology has reached the point where high-level synthesis of VLSI chips and electronic systems is becoming cost effective.
- The high-level synthesis field of research is devoted to problems and solutions related to design automation.

Levels of Abstraction

- We define design synthesis, broadly speaking, as a translation process from a **behavioral description** into a **structural description**.
- To define and differentiate types of synthesis, we use the **Y-chart**, a tripartite representation of design [GaKu83, TLWN90].
- The axes in the Y-chart represent three different domains of description:
 - Behavioral,
 - Structural,
 - physical.
- Along each axis are different levels of the domain of description. As we move farther away from the center of the Y, the level of description becomes more **abstract**.
- We can represent design tools as arcs along a domain's axis or between the axes to illustrate what information is used by each tool and what information is generated, by use of the design tool.

Y-Chart

- We can extend this chart by drawing concentric circles on the Y.
- Each circle intersects the Y axis at a particular level of representation within a domain.
- The circle represents all the information known about the design at some point of time.
- The outer circle is the system level, the next is the microarchitectural or register-transfer (RT) level, followed by the logic and circuit levels.



Levels of Abstraction

- In the **behavioral domain** we are interested in what a design does, not in how it is built.
- We treat the design as one or more **black boxes** with a specified set of **inputs** and **outputs** and a set of functions describing the **behavior** of each output in terms of the inputs over time.
- In addition to stating **functionality**, a behavioral description includes an **interface description** and a **description of constraints** imposed on the design.
- The interface description specifies the **I/O ports** and **timing relationships** or **protocols** among signals at those ports.
- **Constraints** specify technological relationships that must hold for the design to be verifiable, testable, manufacturable and maintainable.

Levels of Abstraction (continued)

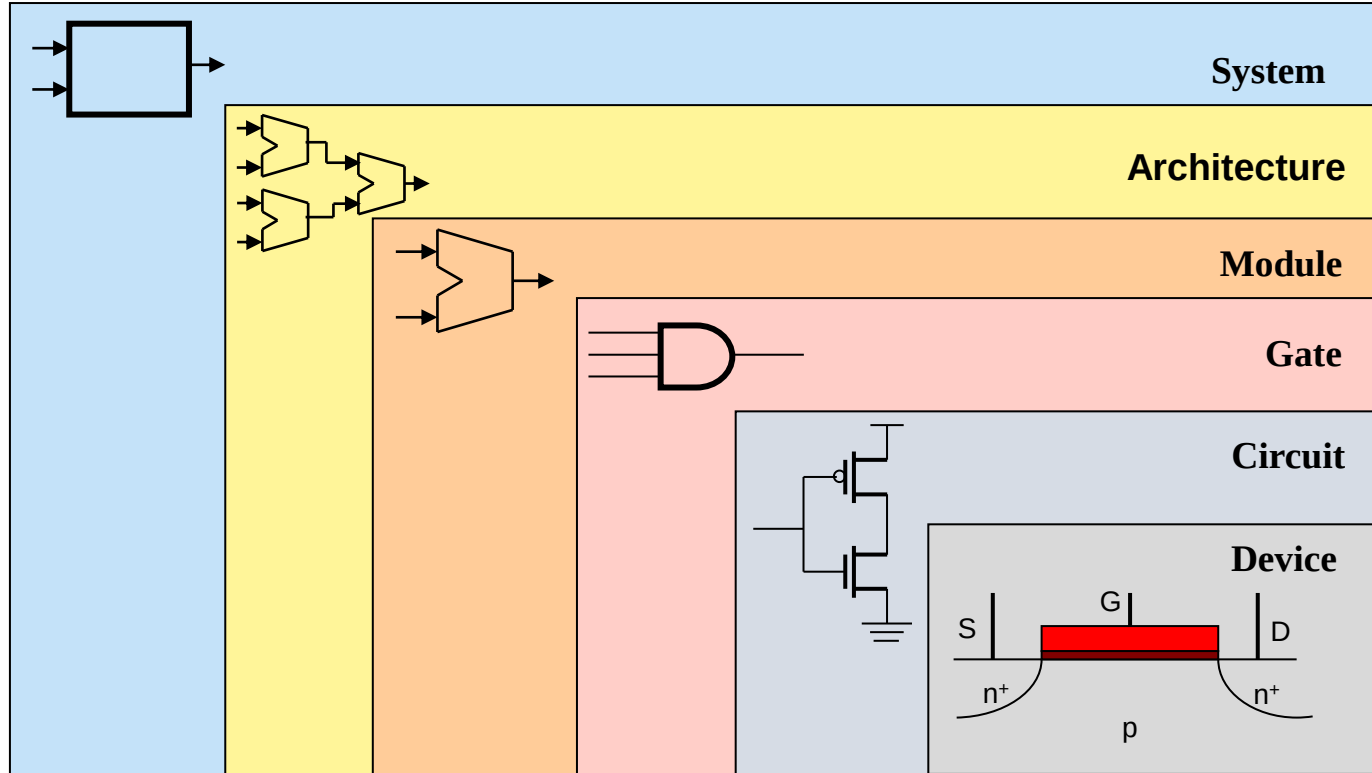
- To **describe behavior**, we use **transfer functions** and **timing diagrams** on the circuit level and **Boolean expressions** and **state diagrams** on the logic level.
- On the **RT level**, time is divided into intervals called **control states** or steps. We use a register-transfer description, which specifies for each control state the condition to be tested, all register transfers to be executed, and the next control state to be entered.
- On the **system level**, we use **variables and language operators** to express functionality of system components. Variables and data structures are not bound to registers and memories, and operations are not bound to any functional units or control states.
- In a **system level** description, **timing** is further abstracted to the order in which variable assignments are executed.

Abstraction Levels

→ Design objects on different abstraction levels.

Level Name	Behavioral Representation	Structural Representation	Physical Representation
System level	Spec. charts Flowcharts Algorithms	Processors Controllers Memories Buses	Cabinets Boards MCMs Chips
Microarchitectural level	Register transfers	ALUs Multipliers MUXs Registers Memories	Chips Floorplans Module floorplans
Logic level	Boolean equations Sequencers	Gates Flip-flops	Modules Cells
Circuit level	Transfer functions	Transistors Connections	Transistor layouts Wire segments Contacts

Levels of Design Abstraction



Levels of Abstraction (continued)

- A **structural** representation bridges the behavioral and physical representation. It is a **one-to-many mapping** of a behavioral representation onto a set of components and connections under constraints such as **cost**, **area**, and **delay**.
- At times, a structural representation, such as a **logic or circuit schematic**, may serve as a **functional description**. On the other hand, behavioral descriptions such as Boolean expressions suggest a trivial implementation, such as a sum-of-product structure consisting of **NOT**, **AND** and **OR** gates.
- The most commonly used levels of structural representation are identified in terms of the basic structural elements used:
 - On the circuit level the basic elements are **transistors**, **resistors**, and **capacitors**,
 - **gates** and **flip-flops** are the basic elements on the logic level.
 - **ALUs**, **multipliers**, **registers**, **RAMs** and **ROMs** are used to identify register-transfers.
 - **Processors**, **memories** and **buses** are used on the system level.

Levels of Abstraction (continued)

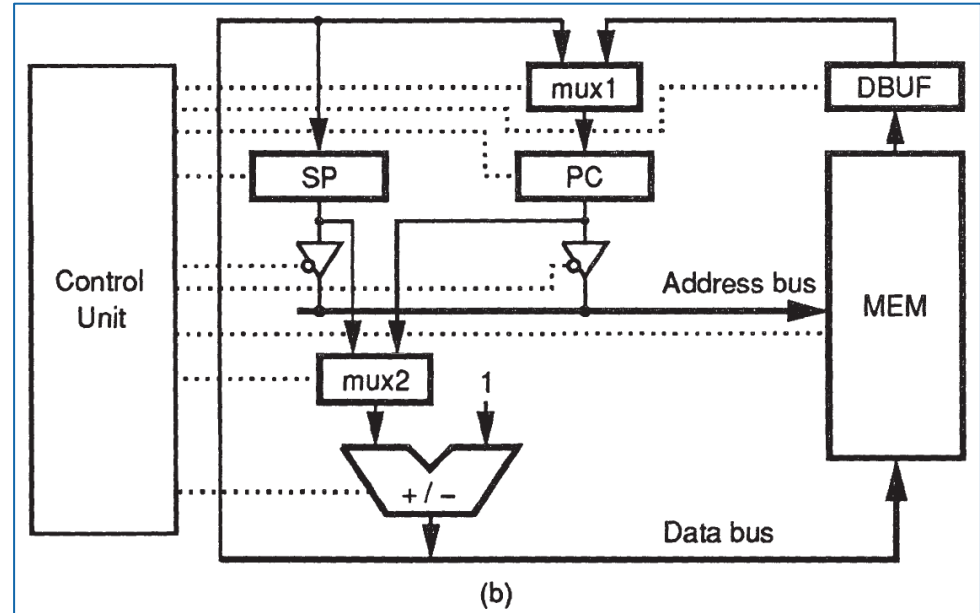
- The **physical representation** ignores, as much as possible, what the design is supposed to do and binds its structure in space or to silicon.
- The most commonly used levels in the **physical representation** are polygons, cells, modules, floorplans, multi-chip modules (MCMs) and printed circuit boards (PCBs).
- Next slide illustrates behavioral, structural, and physical domain descriptions.
- The **behavioral description** shown is a conditional-subroutine-call instruction of a hypothetical processor.
 - In this description, depending on the value of the third bit in the instruction register (IR(3)), either the "then" or "else" branch is executed. In the former case, the program counter (PC) is incremented, and in the latter case, a subroutine address (MEM(PC)) is moved into the PC, after the return address (PC+1) is pushed onto the stack (MEM(SP)).
 - A more complete description of a processor would include all of the machine's instructions and addressing modes.

Levels of Abstraction (continued)

- An example of three domain descriptions: (a) behavioral, (b) structural
- Note that behavioral operators such as + and - in Figure (a) do not necessarily correspond one-to-one to functional blocks in Figure (b).

```
if IR(3) = '0' then
    PC      := PC + 1;
else
    DBUF    := MEM(PC);
    MEM(SP) := PC + 1;
    SP      := SP - 1;
    PC      := DBUF;
end if;
```

(a)

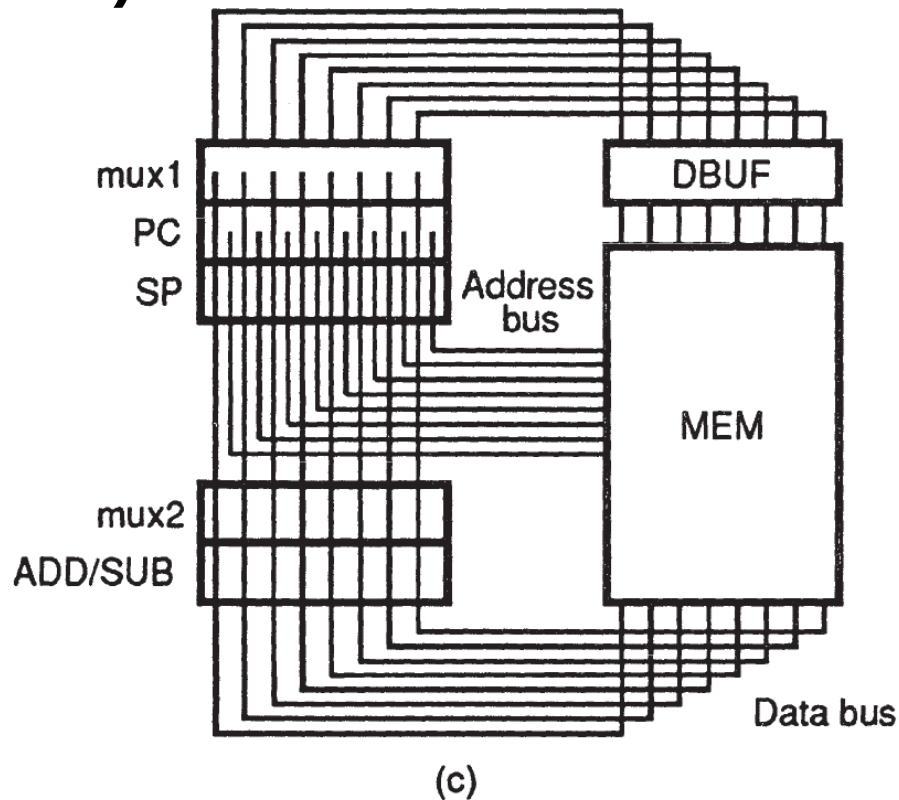


Levels of Abstraction (continued)

- Certainly, the **behavior specified** in the statements must be implemented, but it may be implemented using **shared functional units** that implement several other behavioral operators also.
- We can see that a behavioral description contains very little structural information. At this level of behavioral description, there may be different RT representations that can implement a specific behavior.
- Similarly, a **structural description** contains very little information about **physical representation**. For each structural description there may be many floorplans in the physical domain. One floorplan is shown in Figure (c). Memory (MEM) and its corresponding data buffer (DBUF) occupy the right half of the floorplan, and the rest of the components are grouped together into a datapath laid out as a bit-slice stack with the *Address* and *Data* buses running over each bit slice.

Levels of Abstraction (continued)

- An example of three domain descriptions: (c) physical.
- For each structural description there may be many floorplans in the physical domain.
- In Figure (c) Memory (MEM) and its corresponding data buffer (DBUF) occupy the right half of the floorplan, and the rest of the components are grouped together into a *Datapath* laid out as a bit-slice stack with the *Address* and *Data* buses running over each bit slice.
- Note that extra silicon area is needed whenever a wire or a bus changes direction or makes a "T" connection.



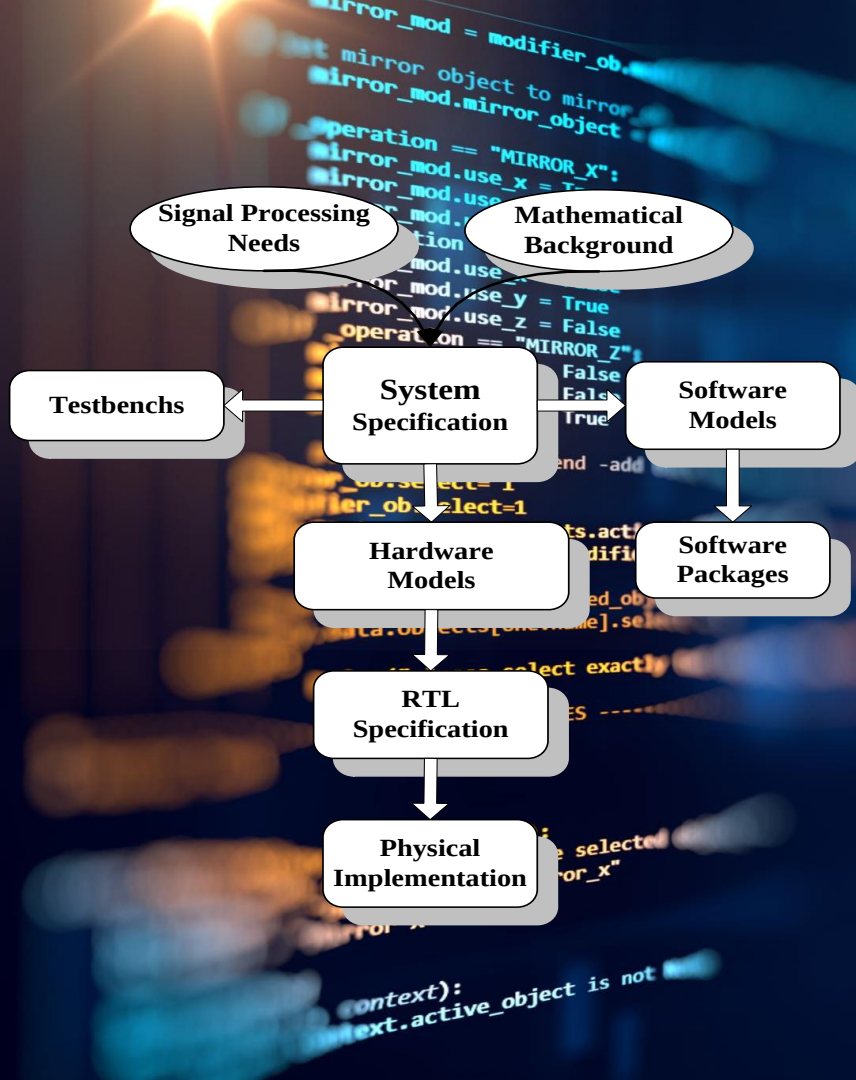
The background of the slide is a complex, futuristic architectural rendering. It features a large, white, hemispherical dome on the right side, which has a grid of lines and a red dotted line graph with diamond markers wrapped around its surface. To the left of the dome are several concentric, circular walkways or platforms. These platforms are decorated with 3D bar charts in red and blue, and small red square markers. The overall aesthetic is high-tech and modern, suggesting a theme of advanced engineering or data visualization.

High Level Synthesis

From Idea to Implementation

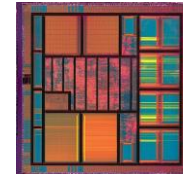
Definition of Synthesis

- The synthesis tasks previously described generate structures that are **not bound in physical space**. At each level of the design, we need another phase of synthesis to add **physical information**. For example:
- **Physical design** or layout synthesis translates a structural description into layout information ready for mask generation.
 - **Cell synthesis** generates a cell layout from a given transistor schematic with specified transistor sizes.
 - **Gate netlists** are converted into modules by placing cells into several rows and connecting I/O pins through routing in the channels between the cells.
 - A **microarchitecture** is converted into chip layout through **floorplanning** with **modules** that represent register-transfer components.
 - **Systems** are usually obtained by placing chips on **multi-chip carriers** or **printed circuit boards** and connecting them through several layers of interconnect.



From Algorithm to Design Implementation

$$X[n] = \frac{1}{2} (x[0] + (-1)^n x[N-1]) + \sum_{i=1}^{N-2} x[i] \cdot \cos \left[\frac{\pi}{N-1} ni \right]$$



Definition of Synthesis

- Synthesis is defined as a translation **from a behavioral description into a structural description**, similar to the compilation of programming languages such as C or Java into an assembly language.
- Each component in the structural description is in turn defined by its own behavioral description.
- The component structure can be obtained through synthesis on a **lower abstraction level**.
- Synthesis, sometimes called **design refinement**, adds an additional level of detail that provides information needed for the next level of synthesis or for manufacturing of the design.
- This more detailed design must satisfy **design constraints** supplied with the original description or generated by a previous synthesis step.

Definition of Synthesis

- Each synthesis step may differ in sophistication, since the work involved is proportional to the **difference in the amount of information** between the **behavioral description** and the **synthesized structural descriptions**.
- For example, a sum-of-products Boolean expression can be trivially converted into a 2-level AND-OR implementation using AND and OR gates with an unlimited number of inputs.
- Substantially more work is needed to implement the same expression with only 2-input NAND gates.
- Although each behavioral description may suggest some trivial implementation, it is generally impossible to find an **optimal implementation** under **arbitrary constraints** for an **arbitrary library of components**.

Definition of Synthesis: Examples

- **Circuit synthesizer** generates a transistor schematic from a set of input-output current, voltage and frequency characteristics or equations.
 - The synthesized transistor schematic should contain **transistor types**, **parameters** and **sizes**.
- **Logic synthesis** translates Boolean expressions into a netlist of components from a given library of logic gates such as NAND, NOR, EXOR, AND-OR-INVERT, and OR-AND-INVERT.
 - In many cases, a structural description using one **library** must be converted into one using **another library**.
 - To do so we convert the first **structural description** into **Boolean expressions**, and then resynthesize it using the second library.

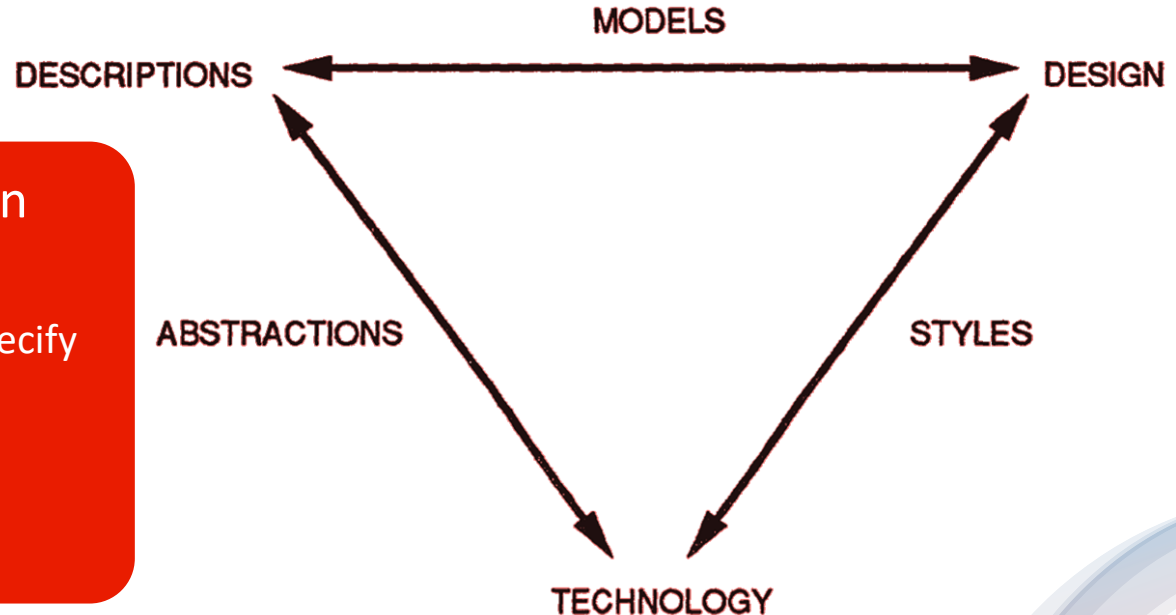
Definition of Synthesis: Examples

- **Register-transfer synthesis** starts with a set of states and a set of register-transfers in each state. One state corresponds roughly to a clock cycle. Register-transfer synthesis generates the corresponding structure in two parts:
 - (a) a **datapath**, which is a structure of storage elements and functional units that performs the given register transfers, and
 - (b) a **control unit** that controls the sequencing of the states in the register transfer description.
- **System synthesis** starts with a set of processes communicating through either shared variables or message passing.
 - It generates a structure of **processors**, **memories**, **controllers** and **interface adapters** from a set of **system components**.
 - Each component can be described by a **register-transfer description**.

Languages, Designs and Technologies

There is a strong correlation between:

- Description languages used to specify a design,
- The design itself, and
- The technology used for implementation of that design



Languages, Designs and Technologies

- **Hardware description languages** (HDL) are used to describe the behavior or structure of systems either on a chip level or board level.
- The target technology introduces a set of **constraints** imposed on the corresponding design implementation.
 - Those constraints may refer to a particular architectural style such as a RISC architecture, to a particular layout methodology such as standard cells, to a particular fabrication process such as CMOS or GaAS, or to a certain component library.
- The **technology constraints** also determine the **quality of the design** and the **time required to finish** it. Similarly, the implementation methodology determines the **CAD tools needed** for design.
- On the other hand, preferred **design styles** (such as pipelining) for a technology can be abstracted into language constructs to provide better understanding and simpler synthesis algorithms.

Languages, Designs and Technologies

- HDLs' constructs should allow an **unambiguous description** for each design.
- In practice, each design can be described or modeled using a single language in several different ways.
- **Example:** next slides show two different descriptions of the same behavior and the different designs derived from each description.
 - We want to design a circuit to compare a *Counter's* output (CNT) with a *Limit* (MIL). An Enable Signal (EN) to control the counter.
 - When asserted, the signal ENIT starts a counter by setting $EN = 1$.
 - When the counter reaches its limit, the comparator asserts its output and sets $EN = 0$, which stops the counter.
- This simple example shows how different modeling practices result in different designs!
- Complex synthesis algorithms are required for disambiguation of the design descriptions.

Languages, Designs and Technologies

```
if CNT  $\neq$  LIM then  
    EN <= ENIT;  
else  
    EN <= '0';  
end if;
```

(a) Level sensitive description

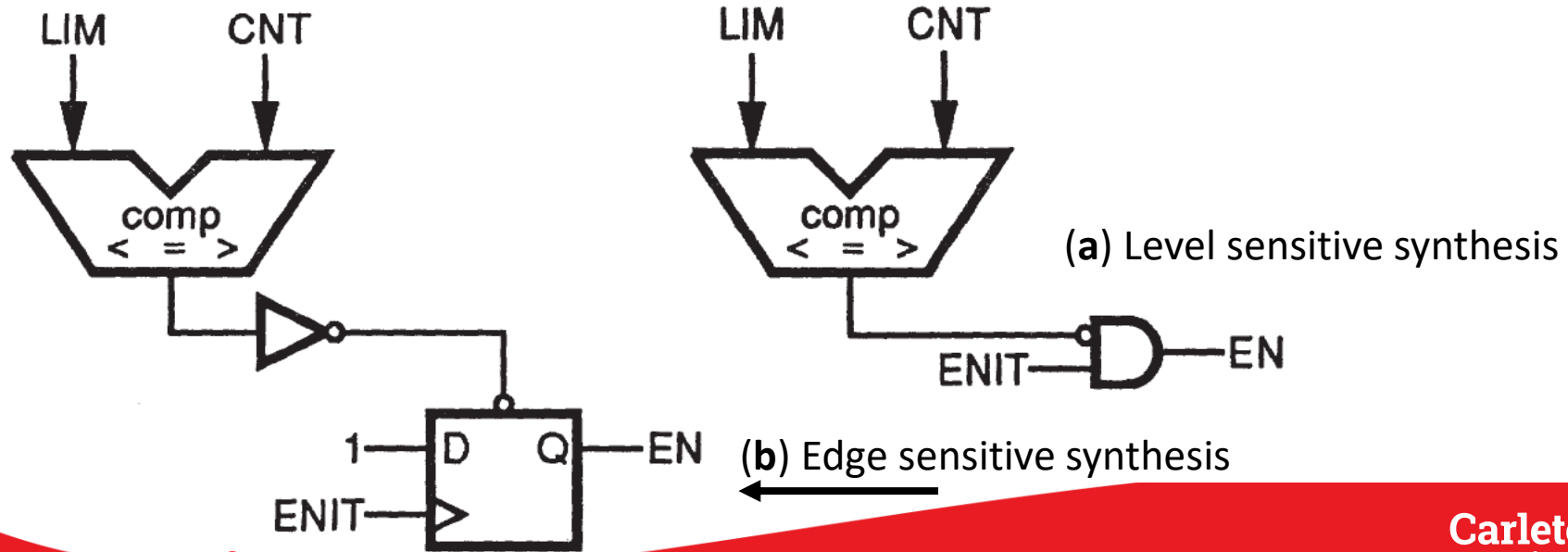
```
if ENIT = '1' and not ENIT'stable then  
    EN <= '1';  
elseif CNT = LIM then  
    EN <= '0';  
end if;
```

(b) Edge sensitive description

- The first model treats ENIT as a **level** signal that stays asserted while the counter is counting.
- The second description uses the positive **edge** of the ENIT signal to set EN = 1 and the output of the comparator to set EN = 0.
- As shown in next slide, this behavior will result in two different implementations.

Languages, Designs and Technologies

- Since the modeler has chosen to use the positive edge of ENIT to indicate the moment when EN becomes equal to 1, the second implementation has an extra D-type flip-flop used to store the occurrence of the positive edge of the ENIT signal.
- **Note:** The implementation shown in Figure (b) is correct but unnecessarily costly.



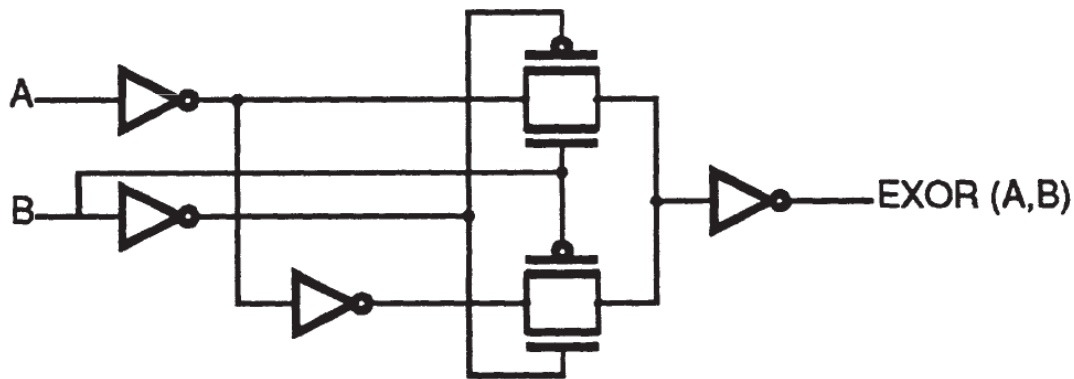
Languages, Designs and Technologies

- In addition to logic level, a **design implementation is not unique** either, i.e., For each function in the design, there are several design styles suitable for different design goals or constraints.
- For **example**, Figures in next slide illustrate two different circuit level implementations of an EXOR function:
 - The design in Figure (a) uses twelve transistors: two for each inverter and additional four transistors for pass-transistor selector.
 - The design in Figure (b) uses only ten transistors: four for the 2-input NOR gate and six for the 2- input AND-NOR gate.
 - The 12-transistor design is better suited for large loads since only 2 output transistors must be oversized.
 - On the other hand, the 10-transistor implementation is better suited for small loads since it will utilize a smaller layout area than the 12-transistor circuit. In the case of large loads, six large transistors at the output will take a much larger area than a 12-transistor circuit.

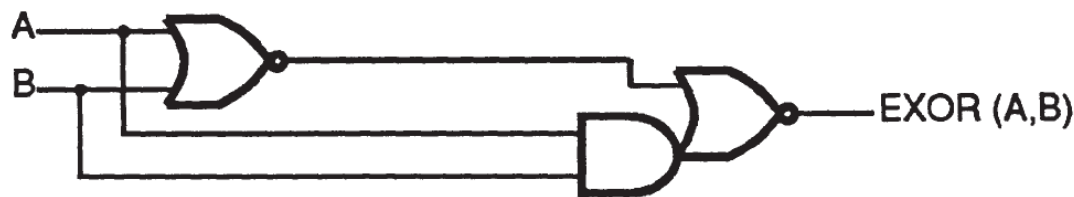
Languages, Designs and Technologies

- Two styles of EXOR implementation using:
- a) transmission gates,
 - b) AND-OR-INVERT gate.

Thus, synthesis systems must allow for making **design tradeoffs** using different design styles for different technologies and design goals.



(a)



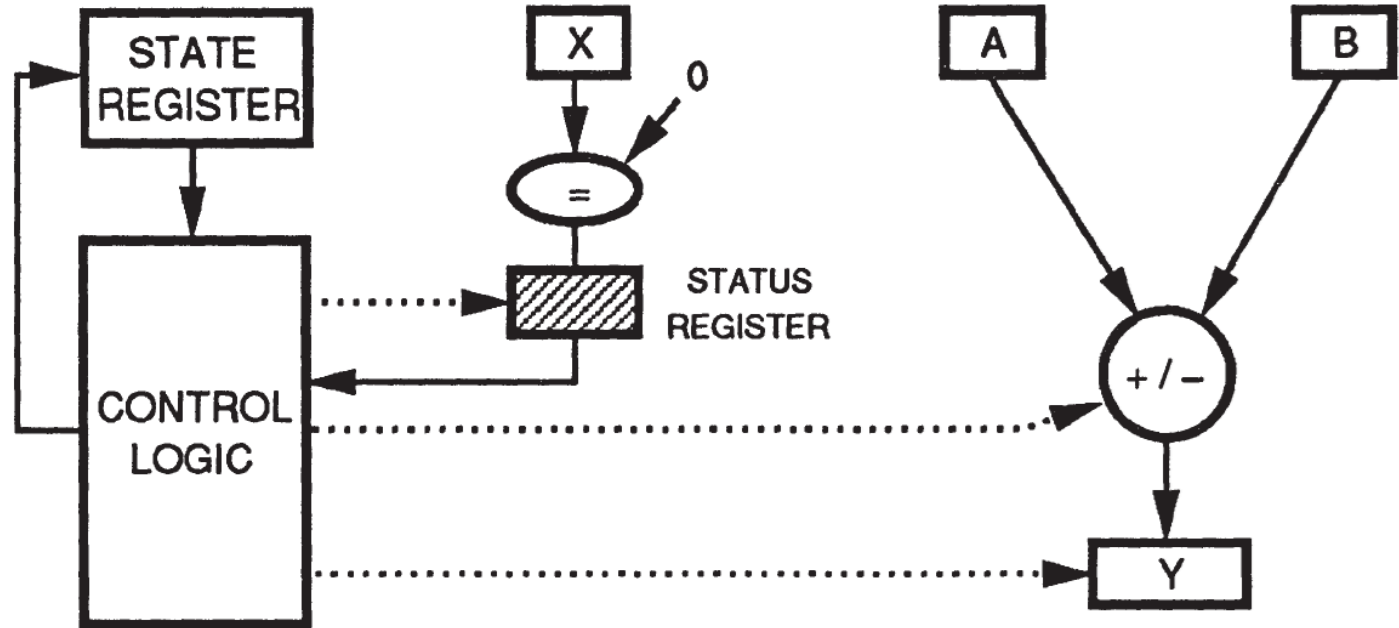
(b)

Languages, Designs and Technologies

- **Design styles** are also reflected in **design descriptions**. The more detailed the design model, the more it suggests a design style.
- Next slide suggests two different styles of datapath implementation of the statement:
$$\textit{if } x = 0 \textit{ then } y = a + b \textit{ else } y = a - b.$$
- The condition $x = 0$ can be directly fed into the control logic or latched in the status register before being tested in the control logic.
- In the former case, the **clock cycle** is longer, but the statement can be executed in one clock cycle. In the latter case, the clock cycle is shorter, and the statement's execution requires two clock periods.
- If the design description allows arbitrary expressions as conditions in each control step, the design style without the status register is more suitable; but if only single bits are allowed as conditions, the style with the status register can be used.

Languages, Designs and Technologies

→ Two different design styles: without status register and with status register (shaded box).



Essential Issues in Synthesis

Challenges:

- Languages and Representations
- Design Modeling
- Design Quality-Measures

Essential Issues in Synthesis

- When synthesis tools are forced to deal with **description models** and **design styles** that do not match, they must incorporate complicated model analysis and **sophisticated style-transformation techniques**.
- Extension of synthesis to the **system level** requires a matching of **models**, **abstractions** and **design styles**.
- HLS uses formalisms based on several different areas, varying from the **programming-language paradigm** for the behavioral description to **layout models** for cost and performance estimation.
- HLS does not have an identifiable formal theory of its own. Most frequently, a mixture of **finite-state machine** and **register-transfer notations**, serve as HLS's formalism.

Essential Issues in Synthesis

- **System design** deals with a large number of different system component types.
- Even for one type of component, such as a processor, there are many versions, each with varieties of options and I/O formats. Among many different types of information needed to describe system components, are:
 - a) the **specification** of RT elements,
 - b) the **connectivity** of declared RT elements,
 - c) the **set of register-transfers** for each state,
 - d) the **sequencing of states** for each combination of control inputs,
 - e) the **physical characteristics** of the components, such as **area**, **power**, **performance** and I/O pin configuration,
 - f) a definition of the **external environment**, including **timing** relationships among input/output **signals**.

Languages and Representations

- The main problem with a design description is its **change with the design over time**. At the beginning, the description is vague, with little or no implementation detail. More detail is added as the design evolves, and the level of **abstraction is lowered** until the design is ready for manufacturing.
- A language that spans all the **levels of abstraction** would be desirable from a management point of view but would be too cumbersome to be mastered by designers, who work only on one or two levels of abstractions.
- Furthermore, different members of the design team require different aspects of a design, for reasons of
 - **verifiability,**
 - **testability,**
 - **reliability,**
 - **manufacturability,**
 - **maintainability,**
 - **manageability.**

Languages and Representations

- Adding all the information needed by those experts to one language known to everybody would produce a very rich language in which information for any particular aspect of the design would be difficult to find.
- Designers need different "views" of the description to suit the application at hand. Thus, the high-level synthesis community needs the capability to describe design views for capture, optimization and verification of different design aspects.
- These languages may be graphical, tabular or textual.

Languages and Representations

- For **example**, if designers want to **balance resource utilization**, they must be able to view or specify the usage of all available resources, such as:
memories, registers, ALUs or buses, on a clock-cycle basis.
- Similarly, as another **example**, designers performing **floorplanning** would be interested in the shape and the size of the same resources. A graphical language for floorplanning that would allow viewing of resource footprints and assignment of their positions on the chip would be necessary for manual optimization of the silicon area.

Design Modeling

- The development of languages for specifying **different aspects of a design** as well as **different formats** for automatic or manual refinement or modification of partially synthesized designs is essential in order for high-level synthesis to be successful.
- Every design description makes a tradeoff between **behavioral** and **structural** information.
- We can describe a design with a single process or break it into several processes. We can continue this **decomposition hierarchically** and generate a hierarchical structure of smaller and smaller components, each in turn described by one process.
- **Note:** there is an essential difference between **behavioral** and **structural** descriptions.
 - Operations in a behavioral description are written **in the order of their occurrence in time**,
 - Operations in a structural description are grouped together according to the **place of execution**.

Design Modeling

- For **example**, a shift operation may be used in three different lines of a long description.
 - All three shift operations are performed by the same shift component in the hardware design.
 - Consequently, the structural description will contain a description of the shift component that is activated at different time during the execution sequence.
- **Note:**
 - A pure **behavioral description** does **not contain explicit information** about the **structure** of the implementation;
 - A pure **structural description** does **not contain explicit information** about the **behavior** of the design over time.
- In some **intermediate descriptions** behavior and structure are intermingled. The number of possible descriptions is even higher in these cases since different language constructs can be used in a design description with a fixed ratio of behavior and structure.

Design Quality-Measures

- In order to assess the quality of a synthesized design, HLS needs good **quality-measures**.
- Counting the number of **functional units**, such as **adders**, **multipliers** and **registers**, although a crude cost measure, is not adequate.
- A good quality-measure must be well focused and point out possible **inefficiencies** or **critical spots** in the design.
- Quality-measures should be computed from the **final synthesized design** description and are used to compare two designs produced from the same specification, however,
 - It is time-consuming to generate completely synthesized designs for comparison of their quality.
 - Hence, more important than the final quality-measures are **estimates of those measures** early during the synthesis process when the final design is not available.

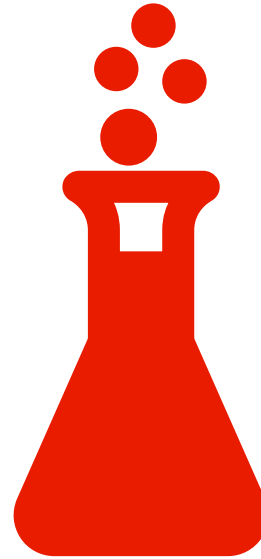
Design Quality-Measures

- **Estimates** must satisfy three criteria:
 - accuracy,
 - fidelity,
 - simplicity.
- Here, **accuracy** is defined in terms of the error between an estimated quality value and the real quality value.
- **Fidelity** is the deviation from the average error over all design points. If an error over all design points is always of the same magnitude, we say that fidelity is high. Estimates with low accuracy but high fidelity are very useful in pruning the design space since they can predict the superiority of one design over another.
- Estimates are evaluated by the **simplicity** of the estimation algorithm that determines the run time of the estimator.

Design Quality-Measures

- Estimation algorithms must be efficient to be useful during the **design-space exploration**.
- **If** the complexity of an estimation algorithms **approaches** the complexities of the synthesis algorithms, then **estimation is useless**, since real designs can be generated as quickly as estimates.
- To be useful, high-level estimates must predict:
 - silicon **area**,
 - wire **delays** and
 - **performance** based on the transistor and wire layout.
- Furthermore, estimates must predict **manufacturing**, **testing** and **maintenance costs**, and must reflect the influence of manufacturing **process variations** on product cost and performance.

High-Level Synthesis Algorithms



High-Level Synthesis Algorithms

→ Again,

Synthesis is a **transformation** of a **behavioral description** into a set of **connected storage and functional units**.

- General types of **algorithms** used in HLS are **partitioning**, **scheduling**, and **allocation**.
- **Partitioning** algorithms deal with the division of a behavioral description or design structure into sub descriptions or substructures in order to reduce the size of the problem or to satisfy some external constraints such as size of the chip, number of pins on the package, power dissipation or maximal wire length.
- **Scheduling** and **allocation** can be thought of as partitioning problems. Scheduling algorithms partition the variable assignments and operations into **time intervals** and allocation partitions them into **storage** and **functional units**.

High-Level Synthesis Algorithms

- **Scheduling** and **allocation** are orthogonal to each other yet are closely intertwined. Scheduling partitions with respect to time, but allocation partitions with respect to hardware resources:
 - If *scheduling is performed before allocation*, it imposes additional constraints on scheduled operations with respect to allocation.
 - For **example**, two operations scheduled in the same time interval cannot be executed by the same functional unit.
 - If *allocation is performed before scheduling*, it restricts the scheduling.
 - For **example**, two operations assigned to the same functional unit can not be executed in the same time interval.

High-Level Synthesis Algorithms

- **Scheduling** and **allocation** are not the only algorithms needed in high-level synthesis.
- After operations are assigned to **time intervals** and **units**, additional algorithms are needed for generation and optimization of
 - **control**,
 - **storage** and **functional units**.
- Traditionally, synthesis of control units is called **sequential synthesis** and synthesis of functional units called **logic synthesis**.
- Storage units can be synthesized with sequential and combinational synthesis methods.

Component Sets and Technology Mapping

- HLS algorithms generate a **structure of (Register-transfer) RT components** such as ALUs, multipliers, registers, counters, and memories.
- These components are given initially as a **library of RTL components**.
- Since components of **different size, functionality and style** (such as carry-ripple vs. carry-look-ahead) are needed, a library of **component generators** to generate component instances on demand would be more appropriate.
- The generators would produce a logic or transistor-level **netlist**, or **layout**, or both for each component.
- Generators are more difficult to develop than designing particular component instances.
- Every designer or design team might develop their own set of commonly used components.

Component Sets and Technology Mapping

- Every designer or design team might develop their own **library**, which is a set of commonly used components.
- As technology and projects change, each team repeats this task over and over again, which makes the **in-house library** grow bigger and bigger!
- Since different component sets may be used from project to project, the problem of **technology mapping** of components from one set to another becomes a critical issue.
- Technology mapping in high-level synthesis is much harder than at the logic level because of the number and variety of components.
- For **example**, there may be several different ALU s performing the same function but with different control encoding, polarity of inputs and outputs, and availability of intermediate carry signals.

Component Sets and Technology Mapping

- Technology mapping from one library to another on the logic level is accomplished by deriving the behavioral description in terms of **Boolean expressions** and resynthesizing it with a new library.
- Deriving a **behavioral description from a structural** one may be difficult, if not impossible, in HLS since a complete behavior of a structural design is not explicitly given. This is more like **reverse engineering**!
- Extending technology mapping to system components such as *DMA controllers*, *I/O interfaces* and *bus arbiters* is even more difficult since no accurate behavioral or structural descriptions exist.
- Furthermore, the problem of mapping one component onto another when they match only in 90% of the functions remains unsolved; in fact, it has yet to be formulated.

Component Sets and Technology Mapping

- At a higher level, a **system database** stores all the information about the system as a **hierarchy** of **communicating processes** or **finite-state machines**.
- It also provides all other information for **testing**, **manufacturing**, **documentation** and **management** of the complete design.
- As mentioned previously, a designer does not want to see all the aspects of the design at one time. A central database will store all aspects of the design and generate different design views on demand. The type and the format of a view should be specified through some format schema.
- The database information is initially obtained from the **original behavioral description** and later augmented by synthesis tools or by a human designer through a graphical interface.
- The database should also check the consistency of upgrades and provide estimates in the case of incomplete information.

Component Sets and Technology Mapping

- Since we cannot expect a completely automated synthesis system and, even if automated, a completely satisfactory design quality, high-level synthesis environments are needed to allow **interactive synthesis**.
- Such a synthesis environment must provide:
 - a) **Interactive mapping** of behavior to structure,
 - b) Generation of **quality-measures** or estimates, and
 - c) **Verification** of changes in behavioral or structural descriptions.
- Interactive synthesis consists of menu-driven design changes or mappings from one representation to another.
- **Quality-measures** provide a comparative capability for an intended design change, and design change **verification validates** the consistency of that change.

Component Sets and Technology Mapping

- For **example**, a variable assignment such as

$$x = a + b + c$$

- executed in a single state can be split into two assignments using a temporary variable *temp*,

$$temp = a + b \quad \text{and} \quad x = temp + c,$$

- each executed in one state.
- In the original case, the implementation needed one state and **two adders** working in parallel, but in the modified implementation only one adder and two states were required.
- The **quality-measures** would indicate that the original implementation was more expensive but faster than the modified implementation and change **verification** would prove that both implementations generate the same result.
- Quality-measures and verification checks can run in the background concurrently with changes performed by the designer.